

The definitive 2026 tech stack for techBuilder

For a multi-tenant construction SaaS targeting Indian SMBs, the optimal stack in 2026 costs \$0–25/month at launch and scales gracefully. The recommended architecture pairs Next.js 16 + Tailwind CSS v4 + shadcn/ui on web with Expo SDK 55 + PowerSync on mobile, backed by NestJS + PostgreSQL with row-level security. Every tool below is free or offers a generous startup tier, chosen specifically for India's mobile-first, offline-heavy, UPI-dominant market. This guide covers 30+ technology decisions across frontend, backend, DevOps, and tooling — with version numbers, pricing, and rationale current to March 2026.

Frontend web: Next.js 16 anchors the dashboard

Next.js 16.2.1 (released March 2026) is the undisputed framework choice. Built on React 19.2, it ships the **React Compiler** (automatic memoization with zero manual effort), **Partial Prerendering** for static shells with streamed dynamic content, and **Turbopack** as a stable Rust-based dev server with filesystem caching. Server Components, Server Actions, and the `cacheLife`/`cacheTag` APIs are all stable — no more `unstable_` prefixes. With ~130K GitHub stars and the largest React ecosystem, Next.js offers the richest middleware, multi-tenant routing, and deployment support available.

Tailwind CSS v4.0 (January 2025) complements it perfectly. The ground-up rewrite uses a **Rust-based Oxide engine** delivering 5× faster full builds and 100×+ faster incremental builds. Configuration moved to CSS-first `@theme` directives — no more `tailwind.config.js`. Zero-config setup means `@import "tailwindcss"` is literally all you need. With ~87K GitHub stars and native integration with shadcn/ui, no alternative (Panda CSS, UnoCSS, StyleX) comes close in ecosystem breadth.

shadcn/ui (CLI v4) is the UI component layer. It's not a traditional library — it's a code distribution system where you copy 50+ accessible components (built on Radix UI primitives) into your project and own them completely. For a SaaS dashboard needing data tables, forms, date pickers, command palettes, and modals, shadcn/ui provides everything out of the box with full Tailwind v4 support. Alternatives like Headless UI (only ~10 components) and Ark UI (smaller community) don't match its breadth.

State, forms, and data flow

The dominant 2026 pattern separates server state from client state:

- **TanStack Query v5** (~5M weekly npm downloads) handles all server state — API data caching, background refetching, optimistic updates, and pagination. Essential for a construction dashboard pulling project data, fleet positions, and attendance records.

- **Zustand v5** (~3KB gzipped, no Provider required) manages client-only state (DEV Community) — sidebar toggles, active tenant context, and UI preferences. (Strapi) It outperforms Redux Toolkit on bundle size by 5× and parse time by 4×. (DEV Community)
- **React Hook Form v7.71 + Zod** handles form state. With **44.6K GitHub stars** and 15M+ weekly downloads, it's battle-tested for complex construction forms (material requests, inspection checklists). Its uncontrolled component approach minimizes re-renders, and shadcn/ui's `<Form>` component is built directly on it. (DEV Community)

Skip Redux Toolkit (too much boilerplate for a startup) and Jotai (Zustand's simpler store model suffices here). (Inziam UI Haq)

Authentication: Clerk for speed, Auth.js for budget

Clerk offers the fastest path to production. Its free tier covers **10,000–50,000 monthly users** (the higher figure reflects a 2026 update), with built-in `<SignIn/>`, (BuildMVPFast) `<OrganizationSwitcher/>`, and multi-tenant Organizations support (SuperTokens) that saves weeks of development. (Supastarter) The catch for India: SMS OTP requires enabling India in the country allowlist, (Clerk) and per-message SMS costs apply. **Pro plan starts at \$25/month.** (BuildMVPFast)

Auth.js v5 (formerly NextAuth) is the zero-cost alternative. Its universal `auth()` function works across Server Components, middleware, and API routes. (Strapi) (Clerk) For Indian OTP login, you'd integrate MSG91 directly — more work but zero vendor lock-in and zero recurring cost. A third option worth watching is **Better-Auth**, an open-source, self-hosted solution with built-in 2FA, RBAC, and passkeys. (Supastarter)

Recommendation: Start with Clerk for fastest launch; migrate to Auth.js + custom OTP if costs become a concern at scale.

Internationalization, real-time, and charts

next-intl (~1.8M weekly downloads, 4× growth in 12 months) is purpose-built for Next.js App Router. (i18nexus) Translations in Server Components add **zero bytes to the client bundle**, (Intlpull) it natively supports Hindi plural rules and ICU message formatting, and setup requires minimal configuration. react-i18next works but adds ~23KB and feels bolted onto the App Router. (Medium)

For **real-time fleet tracking**, start with **Supabase Realtime** (free, unlimited connections) if using Supabase for your database, then graduate to **Socket.io** (self-hosted, **62K+ GitHub stars**) for full bidirectional control. Ably's free tier (6M messages/month) is the best managed alternative.

(BuildMVPFast) WebSockets — not SSE — are necessary since fleet devices must both send locations and receive commands.

For **dashboard charts**, **Recharts v3** (~5M weekly downloads, actively maintained) is the primary choice — declarative JSX API, excellent for project progress and budget tracking. Use **Chart.js**

for performance-heavy views with 10K+ data points (Canvas rendering). **Tremor** (acquired by Vercel, now a copy-paste model like shadcn/ui) ([Vercel](#)) accelerates dashboard assembly with pre-built KPI cards and chart components built on Recharts.

For **web maps**, **Leaflet + React Leaflet** is the zero-cost option (**42K GitHub stars**, BSD license, OpenStreetMap tiles). Add leaflet-draw for geofence/site boundaries and leaflet.heat for attendance heatmaps. **Mapbox GL JS** (50K free map loads/month) ([WP Maps](#)) is the premium upgrade ([SaaS-worthy](#)) with superior WebGL vector rendering and built-in drawing tools. ([Radar](#))

Mobile: Expo SDK 55 with offline-first architecture

Expo SDK 55 (mid-2026) mandates the **New Architecture** — Fabric renderer, TurboModules, and Bridgeless mode with no legacy fallback. ([Expo](#)) It ships React Native 0.83, ([Expo](#)) React 19.x, and Hermes v1 with significant performance gains. ([Expo](#)) **Expo is now the de facto default** for React Native development; Continuous Native Generation eliminates native project management entirely, and config plugins handle all native configuration without ejecting.

EAS Build's free tier provides **15 Android + 15 iOS builds/month** ([expo](#)) with 1,000 MAUs for EAS Update ([Expo Documentation](#)) — sufficient for early development. The Starter plan at **\$19/month** unlocks high-priority builds. ([MetaCTO](#)) Use **EAS Update** for JavaScript-only changes to avoid full native rebuilds.

Offline-first database: PowerSync leads

Construction sites in India routinely lack connectivity. **PowerSync** is the recommended sync engine, ([GitHub](#)) providing automatic bidirectional sync between a client-side SQLite database and your PostgreSQL backend. ([Ignitecookbook](#)) It handles conflict resolution server-side, supports partial sync via Sync Rules, and offers live query subscriptions. ([AWS Marketplace](#)) The self-hosted open edition is **free**; cloud pricing starts at **\$49/month** for 1,000 peak connections. ([powersync](#))

WatermelonDB (v0.27, MIT license) is the zero-cost alternative — it uses SQLite internally with reactive queries and lazy loading, but you must build your own sync endpoints and conflict resolution. Supplement either choice with **react-native-mmkv** for ultra-fast key-value storage (settings, auth tokens) — it's **30× faster than AsyncStorage**.

Avoid MongoDB Realm — it was deprecated in September 2024 and its sync service is being wound down.

Camera, compression, and GPS watermarking

The construction photo documentation stack:

1. **react-native-vision-camera v4.7** captures photos with automatic GPS EXIF tagging via its `enableLocation` prop — no extra libraries needed. `VisionCamera` It supports frame processors via Skia for real-time overlays.
2. **react-native-view-shot** renders a React Native overlay (GPS coordinates, timestamp, project name, worker ID) on top of the photo and captures the composite. `Medium` This proven two-step approach is standard in field documentation apps.
3. **react-native-compressor** `Seven Square` (~50KB APK impact vs FFmpeg's 9MB) reduces 4-5MB construction photos to 200-400KB at quality 0.7-0.8 `npm` — an **80-90% size reduction** critical for uploading over Indian mobile networks.

For **push notifications**, use **Expo Notifications + Firebase Cloud Messaging**. FCM is **completely free with no message limits**, and it's the most reliable channel for Android — which represents ~95% of Indian construction workers' devices. OneSignal's free tier (unlimited mobile push) is an alternative if you need advanced segmentation. `npm` `Engage Lab`

Maps and fleet tracking on mobile

`@rnmapbox/maps` (Mapbox SDK v11) `GitHub` is the clear mobile map choice because it supports **programmatically offline tile downloading** — essential for pre-caching regional maps before workers head to remote construction sites. `Pronteff` The free tier covers **25,000 mobile MAUs/month**, `Radar` `SaaSwothy` with geofencing, custom styling, and 3D terrain. `React Native Experts` Pair with `expo-location` for background GPS tracking. `DEV Community` `react-native-maps` (Google Maps) lacks programmatic offline caching, making it unsuitable for connectivity-constrained Indian sites.

Offline sync architecture

The recommended data flow for offline operation:

- **Data sync:** PowerSync queues writes to local SQLite → syncs to PostgreSQL when online. Reads always hit local SQLite for instant response with no loading spinners. `Ignitecookbook` Conflict resolution follows last-write-wins with server timestamps.
 - **Photo upload queue:** Capture → compress → save to local filesystem → store metadata in SQLite upload queue → background upload with retry when connectivity returns → mark complete → clean up local file.
 - **Background sync:** Use `expo-background-fetch` and `@react-native-community/netinfo` to detect connectivity changes and process queued uploads with exponential backoff and a concurrency limit of 2-3 simultaneous uploads to avoid overwhelming mobile networks.
-

Backend: NestJS + PostgreSQL RLS + Drizzle

NestJS v11 ([Tirnav](#) with v12 planned for Q3 2026) ([GitHub](#)) is the backend framework. Its modular, Angular-inspired architecture maps directly to multi-tenant domain separation ([Tyronneratcliff](#)) — guards and interceptors handle tenant context injection, and built-in WebSocket support ([Tyronneratcliff](#)) covers real-time features. Using the **Fastify adapter** delivers 2–3× throughput over the default Express. With **7,200+ dependent npm packages**, ([npm](#)) it has the largest Node.js framework ecosystem. Hono (v4.12, ~2.8M weekly downloads) is excellent for lightweight edge microservices ([GitHub](#)) but lacks NestJS's dependency injection and structured module system needed for a complex SaaS.

PostgreSQL with shared-schema row-level security is the multi-tenancy strategy. For a startup targeting 100–1,000 tenants, shared schema with a (`tenant_id`) column + RLS policies is simplest and most cost-efficient. Set (`SET LOCAL app.current_tenant`) at the start of each request, ([Techbuddies](#)) and composite indexes on (`(tenant_id, frequently_queried_column)`) keep RLS overhead negligible.

Drizzle ORM (approaching v1.0 stable) edges out Prisma 7 for this project. ([MakerKit](#)) At ~**7.4KB gzipped** ([Bytebase](#)) versus Prisma's ~1.6MB, ([Bytebase](#)) it delivers 20–50ms Lambda cold starts versus Prisma's 80–150ms. ([DEV Community](#)) Critically, Drizzle has **built-in RLS schema support** — you define row-level security policies directly in your TypeScript schema, versus Prisma requiring raw SQL workarounds. The SQL-first query builder gives full control over tenant context commands. ([Prisma](#)) Prisma 7's new lightweight architecture (85–90% bundle reduction from its Rust engine removal) ([DEV Community](#)) makes it a viable alternative if you prioritize its more mature migration tooling. ([MakerKit](#))

File storage: Cloudflare R2 wins on egress

Cloudflare R2 is the storage choice for construction photos and documents. Its **zero egress fees** are transformative for a photo-heavy app ([Substack](#)) — 100GB/month of egress costs \$9 on S3 but \$0 on R2. ([BuildMVPFast](#)) The free tier includes **10GB storage, 1M write operations, and 10M read operations monthly**. ([BuildMVPFast](#)) With 8+ Indian PoPs (Mumbai, Delhi, Chennai, Hyderabad, Kolkata, Bangalore), latency is competitive. R2 uses an S3-compatible API, so standard AWS SDKs work directly. ([Vocal Media](#))

India-specific services: SMS and payments

For **OTP/SMS**, **MSG91** is the primary provider. ([Webxion](#)) It offers prepaid wallet pricing ([G2](#)) at ~**₹0.25/SMS** ([Techjockey](#)) with no monthly commitment, WhatsApp OTP at exact Meta rates with no markup (₹500/month/number after 2 free months), ([G2](#)) and full DLT compliance. **2Factor.in** at **₹0.165/SMS** (pay-per-delivery) ([Smslocal](#)) with 1–3 second delivery speed ([2Factor](#)) serves as a reliable fallback. Twilio costs 3–4× more for Indian numbers. ([Webxion](#)) The strategic play is

WhatsApp OTP as the primary channel (500M+ WhatsApp users in India, higher open rates) with SMS fallback. (Authgear)

For **payments**, **Razorpay** is the unambiguous winner for Indian SaaS:

- **UPI AutoPay** support is essential (Razorpay) — Indian construction SMBs overwhelmingly use UPI, and Stripe India doesn't support UPI for recurring subscriptions
 - ₹0 setup + ₹0 annual maintenance (Cashfree charges ₹4,999/year) (Trade Brains)
 - Built-in Subscriptions API handles all **RBI e-mandate compliance** automatically (pre-debit notifications, smart retries, dunning) (Razorpay)
 - **90%+ payment success rates** via Smart Routing AI versus 70–75% on budget gateways (Trade Brains)
 - Supports 60+ UPI apps including PhonePe, Google Pay, and Paytm (Razorpay)
 - Transaction fee: **2% + GST** (Xflowpay) — the 0.2–0.4% premium over competitors is recovered through higher success rates
-

DevOps: Vercel + Neon + Turborepo at near-zero cost

Vercel is the Next.js hosting default, but note its **Hobby plan restricts commercial use** (CostBench) — techBuilder requires the Pro plan at **\$20/user/month**. (CostBench) Apply for the **Vercel Startup Program** (up to \$30,000 in credits) to offset this. (Checkthat) Vercel offers native wildcard subdomain support for multi-tenant routing, preview deployments on every PR, and edge presence in India.

Cloudflare Pages (\$5/month Workers Paid plan) is the budget alternative with **unlimited bandwidth**, (Hostmeloud) but Next.js 16 support via (@opennextjs/cloudflare) (OpenNext) is less mature than Vercel's native integration. Consider Cloudflare as a CDN layer in front of Vercel to reduce bandwidth costs.

For **PostgreSQL hosting**, **Neon** offers the most developer-friendly free tier: **20 projects, 0.5GB storage each, 5GB egress, scale-to-zero, and database branching** (FreeTiers) for instant dev/staging environment clones. **Supabase** is the all-in-one alternative (500MB DB + 50K MAU auth + 1GB storage free) (MetaCTO) (UI Bakery) but its 2GB egress limit is tight and projects auto-pause after 7 days of inactivity. (UI Bakery +2) Plan for Neon Launch (\$19/month) or Supabase Pro (\$25/month) when moving to production. (Xata)

Monorepo and project structure

Turborepo (rewritten in Rust) with **pnpm workspaces** structures the monorepo: (GitHub)

```
techbuilder/
├─ apps/
│   ├─ web/           # Next.js 16 dashboard
│   ├─ api/           # NestJS backend
│   └─ mobile/        # Expo SDK 55 app
├─ packages/
│   ├─ shared/        # Zod validators, types, constants
│   ├─ ui/            # Shared React components
│   ├─ config-eslint/
│   ├─ config-ts/
│   └─ db/            # Drizzle schema + migrations
```

Turborepo's `--affected` flag (v2.1+) runs tasks only for changed packages, [DEV Community](#) and Remote Caching via Vercel shares build artifacts across CI runs. [GitHub](#) For a startup team, Turborepo's ~20-line `turbo.json` configuration beats Nx's heavier setup. [PkgPulse](#) GitHub Actions' free tier provides **2,000 minutes/month** for private repos [GitHub](#) (unlimited for public) [GitHub +2](#) — use `pnpm turbo run lint test build --affected` to minimize CI time.

For **multi-tenant routing**, use subdomain-based resolution (`acme.techbuilder.com`) via Next.js middleware that extracts the subdomain, resolves the tenant from a cached database lookup, and rewrites the request. [John Kavanagh](#) [Medium](#) Role-based access (Admin → Project Manager → Site Engineer → Worker) layers on top through middleware route maps and Server Component-level authorization checks.

PDF, Excel, and data tables

Hindi PDF rendering is a known pain point. Both jsPDF and `@react-pdf/renderer` have documented Devanagari ligature rendering failures. The only reliable solution is **Puppeteer** in Next.js API routes — it uses Chromium's full text shaping engine for perfect Hindi rendering and captures actual React chart components exactly as displayed. Use it for invoices, daily progress reports, and any document with Hindi content. **@react-pdf/renderer v4.3** (~860K weekly downloads) [npm](#) works for English-only quick exports on the client side. [DEV Community](#) [Medium](#)

For **Excel exports**, **ExcelJS v4.4** (MIT license) is the clear choice over SheetJS because it provides **full cell styling for free** — fonts, colors, borders, conditional formatting, and merged cells. SheetJS Community Edition lacks all styling; its Pro version is paid. [jasvaspring](#) ExcelJS's streaming writer handles large attendance and material datasets efficiently. Supplement with **Papa Parse** (v5.4, ~3.5M weekly downloads) for CSV import/export.

TanStack Table v8 (~26K GitHub stars, MIT license) provides headless, fully customizable data grids. Pair with `@tanstack/react-virtual` for 60FPS virtual scrolling on 50K+ rows and

ExcelJS for export functionality. AG Grid's Enterprise features (\$999/developer/year) [Simple Table](#) and MUI DataGrid's Premium tier (\$588/developer/year) are overkill for a startup — TanStack Table delivers equivalent functionality for free.

Free tools that punch above their weight

PostHog replaces four paid tools in one. Its free tier includes **1M analytics events**, [PostHog](#) **100K error tracking events**, **5K session replays**, and **1M feature flag requests monthly** [PostHog](#) — dwarfing Sentry's 5K errors and 50 replays. [Posthog](#) [PostHog](#) It's fully open-source [PostHog](#) (MIT) and self-hostable. Supplement with **Microsoft Clarity** (completely free, unlimited session replays) if you exceed PostHog's replay limits.

Hoppscotch (**77.6K GitHub stars**, [Checkthat](#) open-source) replaces Postman for API testing — browser-based, no installation, [Better Stack](#) unlimited collections, [Checkthat](#) and supports REST, GraphQL, WebSocket, and SSE. **Figma's free Starter plan** now allows unlimited editors with 3 collaborative design files per team [Saas CRM Review](#) and 500 AI credits/month [Userjot](#) — adequate for MVP design.

Conclusion

The techBuilder stack achieves three critical goals simultaneously: **zero-to-\$25/month launch cost**, **India-market optimization** (offline-first, UPI payments, WhatsApp OTP, regional CDN), and **enterprise-grade multi-tenancy** (PostgreSQL RLS, role-based routing, subdomain isolation). The most consequential decisions are choosing PowerSync over WatermelonDB for offline sync (saves months of custom sync engineering), Razorpay over Stripe for UPI AutoPay subscription billing, and Puppeteer over @react-pdf for Hindi PDF rendering. Every tool recommended is either MIT/Apache-licensed or offers a free tier that covers the first 1,000+ users — the entire stack can reach product-market fit before requiring meaningful infrastructure spend.